

AUDITORIA COMPARATIVA DE PRODUTO

Workix

vs. Emprega São José — Levantamento de Capabilities, Prós e Contras

ESCOPO AVALIADO

**Backend GraphQL, Frontend Vue,
App Android**

DATA

5 de setembro de 2026

PREPARADO PARA

Felipe R. Michetti — Packsys

CONCORRENTE

www.empregasaojose.com.br

MÉTODO

Auditoria de código-fonte (estático)

NATUREZA

Comparativa direta com concorrente

Veredito em uma página

O **Workix** é, no backend, uma plataforma ordens de grandeza mais sofisticada que o Emprega São José: mascaramento real de vagas confidenciais, motor de expiração automática de vagas ("anti-vaga-fantasma") com selo de empresa verificada, busca via Elasticsearch com filtros e facetas que o concorrente simplesmente não tem, currículo normalizado em Markdown com pontuação de completude e sanitização, modelo de consentimento de contato do candidato, Kanban de triagem e agenda de entrevistas nível ATS, motor de faturamento com webhooks idempotentes e tabela de preços pública via API, e uma camada inteira de rede profissional estilo LinkedIn (posts, conexões, grupos, cursos, endossos) que não tem equivalente algum no concorrente.

Mas o usuário final não toca o backend — toca o site e o app. E aí a realidade se inverte: o frontend web tem os formulários de candidatura e de publicação de vaga sem nenhuma submissão real (não enviam dado nenhum), todas as chamadas de API apontam para `localhost:8080` (não funcionam fora da máquina do desenvolvedor), e os filtros de busca são apenas visuais. O app Android tem **9 telas sem o comando que as exibe na tela** (sem `setContentView`) — incluindo a tela de candidatura e a de currículo — e o login tem um atalho que finge sucesso mesmo quando a autenticação real falha. Por fim, encontramos no nosso próprio contador de estatísticas (`statisticsCount`) exatamente o mesmo tipo de bug que é o achado mais grave do concorrente: contar vagas sem filtrar as que já expiraram.

Resumindo: o Emprega São José, apesar de rudimentar e com problemas reais de curadoria de conteúdo, *funciona de ponta a ponta* nas jornadas centrais (buscar, candidatar-se, contatar). O Workix tem um motor muito mais avançado, mas as duas superfícies que o candidato e a empresa realmente usam hoje — site e app — não estão prontas para produção. A prioridade não é mais construir recursos novos: é ligar o que já existe.

1. Escopo e metodologia

Este levantamento comparou três repositórios do ecossistema Workix contra o relatório de auditoria técnica do concorrente Emprega São José (fornecido em PDF, coletado por navegação assistida em 5/9/2026):

- **Backend** — `D:\Packsys\NetBeansProjects\graphql` (Node.js/TypeScript, GraphQL, Sequelize, Elasticsearch, Redis, RabbitMQ) — "projeto maestro" do ecossistema.
- **Frontend Web** — `D:\Packsys\NetBeansProjects\workix-frontend-vue` (Vue 3, Vue Router, Vuex, hospedado no GitHub Pages).
- **App Android** — `D:\Packsys\NetBeansProjects\graphql\android` (Kotlin nativo, Firebase Auth, FCM, GraphQL).

Método: leitura e busca estática de código (`grep/read`) por três agentes de auditoria especializados, sem execução da suíte de testes, sem deploy ao vivo e sem instalação do app em dispositivo.

Limitação importante: diferente da auditoria do concorrente (feita como usuário real em um navegador Chrome contra o site em produção), aqui a análise é de código-fonte. Isso significa duas coisas: (a) alguns comportamentos de runtime podem diferir do que o código estático sugere; (b) não temos hoje um

ambiente Workix "ao vivo" equivalente para auditar da mesma forma como usuário final — isso por si só já é um gap de processo a corrigir antes do lançamento comercial.

2. O que é o Workix (produto)

O Emprega São José é um quadro de vagas regional em WordPress: publica vaga, recebe candidatura, cobra por destaque e liberação imediata. O Workix, pela leitura do código e do `SPECIFICATION.md`, é um projeto de escopo muito maior: uma plataforma de recrutamento tipo ATS (Applicant Tracking System) combinada com uma rede social profissional (modelos de posts, conexões, grupos, cursos, endossos, "social selling score") e um motor de monetização B2B com planos versionados, entitlements por feature e faturamento com nota fiscal (NFS-e). Ou seja, comparamos um quadro de avisos com uma plataforma que se propõe a ser LinkedIn + ATS + marketplace de vagas — o que explica por que a profundidade de dados é tão maior, mas também por que a superfície de produto pronta para uso é bem menor proporcionalmente.

3. Matriz de capabilities comparativa

TEM = implementado e alcançável pelo usuário; **PARCIAL** = existe mas incompleto/decorativo; **NÃO TEM** = ausente; **QUEBRADO** = código existe mas não funciona/não renderiza.

Descoberta e busca de vagas

Capability	Emprega SJ	Workix Backend	Workix Web	Workix Android
Busca por palavra-chave	TEM TEM PARCIAL (stub sem resultado) TEM			
Filtro por cidade/estado	TEM TEM decorativo, sem lógica NÃO TEM			
Filtro por salário	NÃO TEM TEM (range no ES) decorativo (slider morto) NÃO TEM			
Filtro por tipo de contrato	NÃO TEM TEM decorativo TEM (categoria/CLT/PJ)			
Facetas dinâmicas (contagens)	NÃO TEM TEM (jobSearchFacets) NÃO TEM NÃO TEM			
Autocomplete de busca	NÃO TEM TEM NÃO TEM NÃO TEM			
Busca só retorna vagas ativas	NÃO (retorna vagas de 2009) TEM (filtro activated:true no ES) depende do backend depende do backend			
Landing page por cidade (SEO)	TEM NÃO TEM NÃO TEM			

Candidatura

Capability	Emprega SJ	Workix Backend	Workix Web	Workix Android
Candidatar sem cadastro	TEM (3 passos) exige login (JWT/Firebase) exige login exige login			
Upload de currículo em arquivo	NÃO TEM (só colar texto) infra existe, não ligada ao currículo input existe, form não submete NÃO TEM			
Currículo estruturado com pontuação	análise gratuita (ATS score) TEM (completeness score + Markdown sanitizado) não exposto to texto livre			
Candidatura por WhatsApp	TEM NÃO TEM só compartilhar, não candidatar NÃO TEM			

Ponto de entrada de candidatura funciona	QUEBRADO (achado C-1 do concorrente) lógica correta form não submete tela sem setContentView		
Histórico de candidaturas / status	NÃO TEM NÃO TEM implementado mas inalcançável no menu		
Vagas salvas/favoritas	NAO TEM não encontrado NAO TEM NAO TEM		
Alerta de vaga por e-mail	TEM	—	newsletter genérica funcional, não segmentada
Antibot no formulário (captcha/honeypot)	TEM (3 provedores) NAO TEM NAO TEM	—	

Lado da empresa / monetização

Capability	Emprega SJ	Workix Backend	Workix Web
Vaga confidencial com mascaramento	TEM (básico) TEM (entitlement-gated, mascara empresa no resolver) não exposto		
Vaga destacada/patrocinada com rótulo	TEM TEM (JobBoost, is_sponsored, sponsor_label) existe destaque, sem rótulo "patrocinada"		
Moderação/fila de publicação (72h)	TEM não encontrado form de vaga não submete		
Upsell "liberação imediata"	TEM não encontrado	—	
Tabela de preços pública	NAO TEM TEM (query allPlans publica) não exposta na tela		
Página pública de empresa (agrega vagas)	NÃO TEM TEM (company_pages, followers) existe (CompanyView) mas com link de vaga quebrado		
Selo de empresa verificada / taxa de resposta	NAO TEM TEM (response rate 90d automatico) não exposto		
Expiração automática de vaga ("anti-fantasma")	NÃO TEM (achado mais grave dele) TEM (JobExpirationService + outcome obrigatório)	—	
Integração/API para importação em lote	NAO TEM API GraphQL existe, sem endpoint de import em lote dedicado	—	
Faturamento com webhooks idempotentes + auditoria	—	TEM	—

Rede profissional (recurso que o concorrente não tem em nenhum grau)

Modelos confirmados no backend (`src/models/*.ts`): posts de feed, conexões e solicitações de conexão, grupos e posts de grupo, eventos, cursos com aulas e matrículas, hashtags, recomendações de perfil, endosso de habilidades, pontuação de "social selling", mensagens diretas, menções, seguir usuário/empresa, visualizações de perfil, destaque de perfil pago, itens em destaque no perfil, e configuração de marca branca (white-label) completa. **Nenhum desses recursos existe no Emprega São José.** No entanto, no Android a

maior parte está implementada em código real (chat, notificações, kanban) mas **inalcançável** — a navegação principal só tem 4 abas (Início/Vagas/Candidatos/Blog) — e no frontend web nenhum desses módulos tem tela.

Segurança e infraestrutura

Capability	Emprega SJ	Workix
Headers de segurança HTTP (HSTS, CSP, X-Frame-Options, Referrer-Policy)	TEM NÃO TEM em nenhuma camada	
CORS restrito	N/A (WordPress)	origin: '*' liberado
Rate limiting	provável via Cloudflare (não confirmado)	não encontrado
Captcha/honeypot em formulários sensíveis	TEM (Turnstile+reCAPTCHA+hCaptcha+honeypot) NÃO TEM	
API pública bloqueada/protegida	TEM (WP REST responde 401) JWT protege mutations sensíveis; Introspection GraphQL não avaliada	
Sanitização anti-XSS de conteúdo do usuário	N/A	existe só no currículo Markdown, via regex artesanal
Dados estruturados JSON-LD JobPosting (SEO)	TEM (completo) NÃO TEM	
Sitemap/robots/canonical/Open Graph	TEM NÃO TEM (web usa hash-routing sem meta dinâmica)	

4. Pontos fortes do Workix (verificados no código)

+ Vagas confidenciais com mascaramento real e controlado por plano

O resolver de `Job.company` retorna um objeto sintético ("Empresa Confidencial") quando `is_confidential=true` e o usuário não é dono/admin, e a publicação desse tipo de vaga é bloqueada por entitlement de plano (`entitlementsService.can(companyId, 'POST_CONFIDENTIAL_JOBS')`). O concorrente até tem vaga confidencial, mas sem esse nível de controle de acesso por assinatura.

+ Motor "anti-vaga-fantasma" com selo de empresa verificada

`JobExpirationService.autoExpireJobs()` desativa vagas vencidas e exige desfecho obrigatório (`closeJobWithOutcome : HIRED/CANCELLED/CLOSED`). A `CompanyIntegrityService` calcula `response_rate_90d` e concede/revoga selo de verificação automaticamente. Isso ataca exatamente o achado mais grave e mais citado da auditoria do concorrente (contador de vagas inflado por ~197 mil URLs mortas de 2009 nunca desindexadas).

+ Busca com Elasticsearch: filtros, facetas e autocomplete que o concorrente não tem

Filtros por tipo de local de trabalho, tipo de contrato, categoria, senioridade, estado, cidade, skills, faixa salarial, PCD e remoto; ordenação por relevância/recência/salário; facetas dinâmicas com contagem; autocomplete de busca; e toda consulta já filtra `activated:true`, ao contrário da busca do concorrente que devolve milhares de vagas mortas.

+ Currículo normalizado em Markdown com pontuação de completude e sanitização

`NormalizedResume` guarda `completeness_score` calculado (tamanho do conteúdo, presença de resumo, ≥ 3 skills, objetivo de carreira) e passa por sanitização anti-XSS antes de persistir. É funcionalmente equivalente — e mais estruturado — do que a "análise gratuita de currículo com nota" do concorrente.

+ Modelo de consentimento e transparência de contato do candidato

`visibility_settings` dá ao candidato controle sobre ser buscável por recrutadores e aparecer como "open to work"; `contact_unlock` tem campo **obrigatório** `notified_candidate_at`, garantindo que o candidato sempre saiba quando seu contato foi liberado. O concorrente não tem conta de candidato, logo não tem esse conceito.

+ Kanban de triagem e agenda de entrevistas (nível ATS)

Etapas de kanban por vaga, cartões de candidato com histórico auditável de movimentação entre etapas, e agenda de entrevistas com formato, duração, local/link e status. O concorrente é um quadro de avisos simples — não tem nenhum recurso de gestão de processo seletivo.

+ Monetização madura: planos, entitlements, faturamento com auditoria e preço público

Modelo de planos versionados com features e limites (`PlanFeature`), serviço central de entitlements (`can()`), webhooks de pagamento idempotentes (checagem por `gateway_event_id` único) com log de auditoria completo (`before_json` / `after_json`), e — diferente do concorrente, que não publica preço nenhum dos upsells — uma query `allPlans` pública que expõe todos os planos e preços sem autenticação.

+ Camada de rede profissional (posts, conexões, grupos, cursos, endossos, white-label)

Um produto inteiro que o Emprega São José simplesmente não tem categoria equivalente: feed de posts, conexões entre usuários, grupos com posts, eventos, mini-plataforma de cursos, endosso de habilidades, pontuação de "social selling", mensagens diretas e configuração de marca branca para revenda B2B do produto.

+ App nativo com notificação push (o concorrente não tem app)

Canal de notificação Android configurado via Firebase Cloud Messaging, aparecendo na bandeja do sistema mesmo com o app fechado — capacidade que um site responsivo, por melhor que seja, não replica. (Ver ressalva na seção de fragilidades: o encanamento do token ainda não está completo.)

5. Fragilidades e achados técnicos

Organizados por severidade, no mesmo espírito da auditoria do concorrente — mas com uma diferença importante a registrar: lá, o próprio auditor concluiu que "nenhum achado é falha de segurança, a base técnica é sólida, falta curadoria de conteúdo". Aqui, vários achados abaixo são **falhas de integridade funcional** nas duas superfícies que o usuário toca (site e app) — não apenas higiene de conteúdo.

C-1 — Formulários web de candidatura e publicação de vaga não enviam dados

CRÍTICO

`PostResumeView.vue` e `PostJobView.vue` não têm `@submit`, os campos não têm `v-model`, e o botão final é um link `<a href="#">` sem handler algum. Hoje, ninguém consegue se candidatar nem publicar vaga pelo site, mesmo preenchendo o formulário inteiro.

C-2 — Todas as chamadas de API do frontend web apontam para

`localhost:8080`

CRÍTICO

34 ocorrências em 24 arquivos. Em produção (`www.workix.com.br`, GitHub Pages), nenhuma chamada de API funcionaria — login, cadastro, listagem de vagas, tudo aponta para a máquina local do desenvolvedor, apesar de já existir `.env.default` não utilizado nesses pontos.

C-3 — 9 telas do Android sem

`setContentView`

(tela em branco/crash)

CRÍTICO

`RegisterActivity`, `JobDetailActivity`, `PostJobActivity`, `CandidateDetailActivity`, `PostResumeActivity`, `CompanyDetailActivity`, `BlogPostActivity`, `InterviewsActivity`, `RecruitmentKanbanActivity` constroem widgets em código mas nunca os anexam à tela. Isso inclui a tela de **candidatura à vaga** e a de **currículo** — os dois fluxos centrais do produto.

C-4 — Login/cadastro Android finge sucesso mesmo com falha real de autenticação

CRÍTICO

`AuthViewModel.kt` cai em fallback que fabrica um UID e token falsos (`"dev-fb-uid-..."`) e reporta sucesso ao usuário mesmo se o Firebase ou a sincronização com o backend falharem — mascarando erros reais e criando sessões "fantasma".

C-5 — Contador de estatísticas conta vagas sem filtrar as expiradas (mesmo bug do concorrente)

CRÍTICO

`stats.repo.ts` faz `Job.count()` sem filtro de `activated / outcome_status`. É exatamente a classe de bug apontada como o achado mais grave da auditoria do concorrente (contador de "vagas ativas" inflado por incluir histórico morto).

A-1 — Ausência de headers de segurança HTTP e CORS aberto

ALTO

Sem helmet/HSTS/CSP/X-Frame-Options/Referrer-Policy/Permissions-Policy em nenhuma camada; `src/index.ts` libera CORS com `origin: '*'`. O concorrente, apesar de ser um WordPress simples, tem todos esses cabeçalhos configurados corretamente.

A-2 — Sem rate limiting e sem captcha/honeypot em nenhum formulário/mutation

ALTO

Nenhuma ocorrência de rate-limit, honeypot, reCAPTCHA/hCaptcha/Turnstile em todo o backend, frontend web ou app. O concorrente usa três provedores de captcha diferentes nos pontos sensíveis.

A-3 — Sem dados estruturados JSON-LD JobPosting nem infraestrutura de SEO

ALTO

Nenhuma geração de schema.org JobPosting; frontend web sem sitemap, robots.txt, canonical, Open Graph/Twitter Card, ou meta tags dinâmicas — e ainda usa hash-routing (`#/vagas`), historicamente pior para indexação. É exatamente o ponto mais forte do concorrente e hoje não temos equivalente algum.

A-4 — Filtros de busca do site são inteiramente decorativos

ALTO

`JobsList.vue` tem UI completa de filtro (nível, modelo de trabalho, tipo, cidade, slider de salário), mas nenhum input tem `v-model`, `name` ou handler — são elementos herdados do template visual original, sem lógica JavaScript nenhuma por trás, mesmo o backend já suportando todos esses filtros via Elasticsearch.

A-5 — Kanban e Entrevistas no Android nem estão registrados no manifest

ALTO

`RecruitmentKanbanActivity` e `InterviewsActivity` causariam `ActivityNotFoundException` se qualquer código tentasse abri-las — o que hoje nada faz. É recurso "de papel": existe no repositório, não existe para o usuário.

M-1 — Cobertura real de testes abaixo da meta configurada

MÉDIO

Meta de 100% no `jest.config.js`; cobertura real documentada em `KNOW_ISSUES.md` (ISSUE-001) fica em ~83-91%, com débito concentrado em módulos legados (posts, messaging, resumes, selective_processes, stats, subscribers, testimonials, users).

M-2 — Sanitização de Markdown do currículo é regex artesanal

MÉDIO

Remove tags/atributos perigosos por expressão regular fixa, não por biblioteca madura (DOMPurify/sanitize-html) — risco real de bypass não coberto pelas regras estáticas.

M-3 — Verificação de JWT engole erros silenciosamente

MÉDIO

`extractJWTMiddleware` apenas segue adiante sem popular o contexto de usuário quando o token é inválido, sem log — dificulta detectar tentativas de ataque.

M-4 — Sem upload de currículo em arquivo em nenhuma camada, mas o site promete

MÉDIO

O backend tem infraestrutura genérica de upload (validação por magic bytes) mas não conectada ao currículo; o app Android só aceita texto; e o site tem um campo de upload de PDF na tela que não está ligado a nada (form não submete) — pior que apenas "não ter", porque promete visualmente algo que não entrega.

M-5 — Link de vaga na página da empresa (web) aponta para rota errada

MÉDIO

`CompanyView.vue` aponta para `/vagas?id=` em vez de `/detalhes_vaga?id=`, levando à lista paginada ignorando o id, em vez do detalhe da vaga específica.

M-6 — Sem canal de candidatura/notificação via WhatsApp em nenhuma camada

MÉDIO

O site só usa WhatsApp para compartilhamento social, não para candidatura ou contato — recurso de distribuição forte do concorrente que não replicamos.

M-7 — Token de push (FCM) nunca chega ao backend

MÉDIO

`onNewToken` no Android está vazio (só comentário) — a notificação push está configurada no papel, mas o direcionamento real de notificações por usuário não funciona.

M-8 — Android mistura três clientes de rede diferentes

MÉDIO

Um cliente GraphQL manual (o único de fato usado), um cliente Apollo declarado e nunca referenciado (código morto), e uma terceira camada Retrofit REST usada em outras telas — arquitetura inconsistente que aumenta custo de manutenção.

M-9 — Grande parte da "rede profissional" do Android está implementada mas inalcançável

MÉDIO

Chat, notificações e "minhas candidaturas" têm código funcional real e completo, mas a navegação principal só tem 4 abas — nenhuma leva a essas telas. É investimento de engenharia pago e não entregue ao usuário.

B-1 — Textos e dados de exemplo (lorem ipsum, empresas fictícias em inglês) visíveis em produção

BAIXO

`JobDetailsView.vue` mostra texto latim fixo quando campos vêm vazios;
`Jobs2View.vue` / `Candidates2View.vue` são telas 100% mockadas em inglês, acessíveis pelo menu — mesma classe de bug de "acabamento" citada na auditoria do concorrente (textos em inglês, formato americano).

B-2 — Sem Material Design 3, sem modo escuro, sem `contentDescription` no Android **BAIXO**

Tema ainda em AppCompat clássico; nenhuma ocorrência de `contentDescription` em toda a UI nova — acessibilidade não foi considerada nas telas recém-construídas.

B-3 — Tabela de preços já existe na API mas não é exposta em nenhuma tela **BAIXO**

A query pública `allPlans` já traz planos e preços, mas nem o site nem o app expõem essa tela — oportunidade perdida de reforçar o próprio "Pacto de Transparência" #2 declarado no `SPECIFICATION.md` do projeto.

6. Recomendações priorizadas

#	Ação	Esforço	Por quê
1	Adicionar <code>setContentView</code> nas 9 Activities Android sem UI renderizada	Baixo/Médio	Sem isso, candidatura e currículo não abrem no app
2	Ligar os formulários web de candidatura, publicação de vaga e cadastro de empresa a chamadas reais de API	Médio	Hoje é impossível se candidatar ou publicar vaga pelo site
3	Substituir as 34 ocorrências de <code>localhost:8080</code> por variável de ambiente no frontend web	Baixo	O site não funciona em produção sem isso
4	Remover o fallback de login "fake success" do Android	Baixo	Evita sessões fantasma e falsa sensação de sucesso de autenticação
5	Corrigir <code>statisticsCount</code> para filtrar vagas ativas/desfecho	Baixo	Mesmo bug crítico de contador inflado do concorrente
6	Adicionar headers de segurança (helmet), travar CORS, rate limiting e captcha/honeypot nos formulários sensíveis	Médio	Paridade mínima de segurança com o concorrente
7	Implementar JSON-LD JobPosting, sitemap, robots.txt e Open Graph/Twitter Card no site	Médio/Alto	Maior ponto forte de SEO do concorrente; hoje não temos nada equivalente
8	Ligar as telas Android já prontas (notificações, minhas candidaturas, chat) à navegação principal	Baixo	Trabalho de engenharia já pago, só falta expor ao usuário
9	Terminar (ou remover do menu) o Kanban de Recrutamento e a Agenda de Entrevistas no Android	Médio	Hoje causariam crash se alguém tentasse abri-los
10	Expor tela de planos/preços no site e app a partir da query pública <code>allPlans</code>	Baixo	Reforça o Pacto de Transparência da própria especificação e é diferencial real vs. concorrente
11	Ativar filtros de busca reais no site (salário, contrato, cidade) usando os parâmetros que o Elasticsearch já aceita	Médio	Backend já suporta; frontend só finge que suporta
12	Avaliar canal de candidatura/notificação via WhatsApp	Médio/Alto	Recurso forte do concorrente que hoje não replicamos em nenhuma camada
13	Fechar o gap de cobertura de testes real vs. meta de 100% já configurada	Médio	Reduz risco de regressão nos módulos legados listados no KNOW_ISSUES
14	Consolidar o Android em um único cliente de rede (remover Apollo morto e mistura REST/GraphQL)	Médio	Reduz dívida técnica de arquitetura

Relatório produzido em 5 de setembro de 2026 a partir de auditoria estática de código-fonte dos repositórios `graphql`, `workix-frontend-vue` e `graphql/android`, comparados ao relatório de auditoria técnica do concorrente Emprega São José (coleta em 5/9/2026, navegação assistida). Nenhum ambiente de produção Workix foi acessado como usuário final nesta rodada — recomenda-se repetir a auditoria como usuário real assim que o site/app estiverem em produção estável.